

Hash function	Reference	Status	Restrictions
SHA-512/224	FIPS 180-4	L	Not suitable for HMAC, KDF, RBG, digital signatures
SHA3-224	FIPS 202	L	Not suitable for HMAC, KDF, RBG, digital signatures

Hash function	Reference	Status	Restrictions
SHA-1	RFC 3174 & RFC 6194	L	Not suitable for HMAC, KDF, RBG, digital signatures
CRC (any length)		D	
MD4	RFC 1320	D	
MD5	RFC 1321	D	

Hash Functions for Password Storage

For secure password hashing, dedicated hash functions must be used. These slow-hashing algorithms mitigate brute-force and dictionary attacks by increasing the computational difficulty of password cracking.

KDF	Reference	Required Parameters	Status
argon2id	RFC 9106	$t = 1: m \geq 47104$ (46 MiB), $p = 1$ $t = 2: m \geq 19456$ (19 MiB), $p = 1$ $t \geq 3: m \geq 12288$ (12 MiB), $p = 1$	A
scrypt	RFC 7914	$p = 1: N \geq 2^{17}$ (128 MiB), $r = 8$ $p = 2: N \geq 2^{16}$ (64 MiB), $r = 8$ $p \geq 3: N \geq 2^{15}$ (32 MiB), $r = 8$	A
bcrypt	A Future-Adaptable Password Scheme	$\text{cost} \geq 10$	A
PBKDF2-HMAC-SHA-512	NIST SP 800-132, FIPS 180-4	$\text{iterations} \geq 210,000$	A

KDF	Reference	Required Parameters	Status
PBKDF2- HMAC- SHA- 256	NIST SP 800-132, FIPS 180-4	iterations $\geq 600,000$	A
PBKDF2- HMAC- SHA-1	NIST SP 800-132, FIPS 180-4	iterations $\geq 1,300,000$	L

Approved password-based key derivations functions can be used for password storage.

Key Derivation Functions (KDFs)

General Key Derivation Functions

KDF	Reference	Status
HKDF	RFC 5869	A
TLS 1.2 PRF	RFC 5248	L
MD5-based KDFs	RFC 1321	D
SHA-1-based KDFs	RFC 3174 & RFC 6194	D

Password-based Key Derivation Functions

KDF	Reference	Required Parameters	Status
argon2id	RFC 9106	t = 1: m ≥ 47104 (46 MiB), p = 1 t = 2: m ≥ 19456 (19 MiB), p = 1 t = 3: m ≥ 12288 (12 MiB), p = 1	A
scrypt	RFC 7914	p = 1: N $\geq 2^{17}$ (128 MiB), r = 8 p = 2: N $\geq 2^{16}$ (64 MiB), r = 8 p = 3: N $\geq 2^{15}$ (32 MiB), r = 8	A
PBKDF2- HMAC- SHA- 512	NIST SP 800-132, FIPS 180-4	iterations $\geq 210,000$	A

KDF	Reference	Required Parameters	Status
PBKDF2- HMAC- SHA- 256	NIST SP 800-132, FIPS 180-4	iterations $\geq 600,000$	A
PBKDF2- HMAC- SHA-1	NIST SP 800-132, FIPS 180-4	iterations $\geq 1,300,000$	L

Key Exchange Mechanisms

This section provides additional information for V11.6 Public Key Cryptography.

KEX Schemes

A security strength of 112 bits or above MUST be ensured for all Key Exchange schemes, and their implementation MUST follow the parameter choices in the following table.

Scheme	Domain Parameters	Forward Secrecy	Status
Finite Field Diffie-Hellman (FFDH)	$L \geq 3072$ & $N \geq 256$	Yes	A
Elliptic Curve Diffie-Hellman (ECDH)	$f \geq 256-383$	Yes	A
Encrypted key transport with RSA-PKCS#1 v1.5		No	D

Where the following parameters are:

- k is the key size for RSA keys.
- L is the size of the public key and N is the size of the private key for finite field cryptography.
- f is the range of key sizes for ECC.

Any new implementation MUST NOT use any scheme that is NOT compliant with NIST SP 800-56A & B and NIST SP 800-77. Specifically, IKEv1 MUST NOT be used in production.

Diffie-Hellman groups

The following groups are approved for implementations of Diffie-Hellman key exchange. Security strengths are documented in NIST SP 800-56A, Appendix D, and NIST SP 800-57 Part 1 Rev.5.

Group	Status
P-224, secp224r1	A
P-256, secp256r1	A
P-384, secp384r1	A
P-521, secp521r1	A
K-233, sect233k1	A
K-283, sect283k1	A
K-409, sect409k1	A
K-571, sect571k1	A
B-233, sect233r1	A
B-283, sect283r1	A
B-409, sect409r1	A
B-571, sect571r1	A
Curve448	A
Curve25519	A
MODP-2048	A
MODP-3072	A
MODP-4096	A
MODP-6144	A
MODP-8192	A
ffdhe2048	A
ffdhe3072	A
ffdhe4096	A
ffdhe6144	A
ffdhe8192	A

Message Authentication Codes (MAC)

Message Authentication Codes (MACs) are cryptographic constructs used to verify the integrity and authenticity of a message. A MAC takes a message and a secret key as inputs and produces a fixed-size tag (the MAC value). MACs are widely used in secure communication protocols (e.g., TLS/SSL) to ensure that messages exchanged between parties are authentic and intact.

MAC			
Algorithm	Reference	Status	Restrictions
HMAC-SHA-256	RFC 2104 & FIPS 198-1	A	
HMAC-SHA-384	RFC 2104 & FIPS 198-1	A	
HMAC-SHA-512	RFC 2104 & FIPS 198-1	A	
KMAC128	NIST SP 800-185	A	
KMAC256	NIST SP 800-185	A	
BLAKE3 (keyed_hash mode)	BLAKE3 one function, fast everywhere	A	
AES-CMAC	RFC 4493 & NIST SP 800-38B	A	
AES-GMAC	NIST SP 800-38D	A	
Poly1305-AES	The Poly1305-AES message-authentication code	A	
HMAC-SHA-1	RFC 2104 & FIPS 198-1	L	
HMAC-MD5	RFC 1321	D	

Digital Signatures

Signature schemes MUST use approved key sizes and parameters per NIST SP 800-57 Part 1.

Signature Algorithm	Reference	Status
EdDSA (Ed25519, Ed448)	RFC 8032	A
XEdDSA (Curve25519, Curve448)	XEdDSA	A
ECDSA (P-256, P-384, P-521)	FIPS 186-4	A
RSA-RSSA-PSS	RFC 8017	A

Signature Algorithm	Reference	Status
RSA-SSA-PKCS#1 v1.5	RFC 8017	D
DSA (any key size)	FIPS 186-4	D

Post-Quantum Encryption Standards

PQC implementations must be in line with FIPS-203/204/205 as there is minimal hardened code nor implementation reference yet. <https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards>

The proposed mlkem768x25519 post-quantum hybrid TLS key agreement method is supported by major browsers such as Firefox release 132 and Chrome release 131. It may be used in cryptographic testing environments or when available within industry- or government-approved libraries.

Appendix D: Recommendations

Introduction

Whilst preparing version 5.0 of the Application Security Verification Standard (ASVS), it became clear that there were a number of existing and newly suggested items that shouldn't be included as requirements in 5.0. This may have been because they were not in scope for ASVS as per the definition for 5.0 or alternatively it was felt that while they were a good idea, they could not be made mandatory.

Not wanting to lose all these items entirely, some have been captured in this appendix.

Recommended, in-scope mechanisms

The following items are in-scope for ASVS. They should not be made mandatory but it is strongly recommended to consider them as part of a secure application.

- A password strength meter should provided to help users set a stronger password.
- Create a publicly available security.txt file at the root or .well-known directory of the application that clearly defines a link or e-mail address for people to contact owners about security issues.
- Client-side input validation should be enforced in addition to validation at a trusted service layer as this provides a good opportunity to discover when someone has bypassed client-side controls in an attempt to attack the application.
- Prevent accidentally accessible and sensitive pages from appearing in search engines using a robots.txt file, the X-Robots-Tag response header or a robots html meta tag.
- When using GraphQL, implement authorization logic at the business logic layer instead of the GraphQL or resolver layer to avoid having to handle authorization on every separate interface.

References:

- More information on security.txt including a link to the RFC

Software Security principles

The following items were previously in ASVS but are not really requirements. Rather they are principles to consider when implementing security controls that when followed will lead to more robust controls. These include:

- Security controls should be centralized, simple (economy of design), verifiably secure, and reusable. This should avoid duplicate, missing, or ineffective controls.
- Wherever possible, use previously written and well-vetted security control implementations rather than relying on implementing controls from scratch.
- Ideally, a single access control mechanism should be used to access protected data and resources. All requests should pass through this single mechanism to avoid copy and paste or insecure alternative paths.
- Attribute or feature-based access control is a recommended pattern whereby the code checks the user's authorization for a feature or data item rather than just their role. Permissions should still be allocated using roles.

Software Security processes

There are a number of security processes which were removed from ASVS 5.0 but are still a good idea. The OWASP SAMM project may be a good source for how to effectively implement these processes. The items which were previously in ASVS include:

- Verify the use of a secure software development lifecycle that addresses security in all stages of development.
- Verify the use of threat modeling for every design change or sprint planning to identify threats, plan for countermeasures, facilitate appropriate risk responses, and guide security testing.
- Verify that all user stories and features contain functional security constraints, such as "As a user, I should be able to view and edit my profile. I should not be able to view or edit anyone else's profile"
- Verify availability of a secure coding checklist, security requirements, guideline, or policy to all developers and testers.
- Verify that an ongoing process exists to ensure that the application source code is free from backdoors, malicious code (e.g., salami attacks, logic bombs, time bombs), and undocumented or hidden features (e.g., Easter eggs, insecure debugging tools). Complying with this section is not possible without complete access to source code, including third-party libraries, and is therefore probably only suitable for applications requiring the very highest levels of security.

- Verify that mechanisms are in place to detect and respond to configuration drift in deployed environments. This may include using immutable infrastructure, automated redeployment from a secure baseline, or drift detection tools that compare current state against approved configurations.
- Verify that configuration hardening is performed on all third-party products, libraries, frameworks, and services as per their individual recommendations.

References:

- OWASP Threat Modeling Cheat Sheet
- OWASP Threat modeling
- OWASP Software Assurance Maturity Model Project
- Microsoft SDL

Appendix E - Contributors

We gratefully acknowledge the contributions of the following people who have commented or opened pull requests since the the release of ASVS 4.0.0.

If you are aware of any mistakes or would like your name to appear differently, please let us know.

Johan Sydseter (sydseter)	luis servin (lfservin)	Oleksii Dovydkov (oleksiidov)	IZUKA Masahiro (maizuka)
James Sulinski (jsulinski)	Eli Saad (ThunderSon)	kkshitish9	Andrew van der Stock (vanderaj)
Rick M (kingthorin)	Bankde Eakasit (Bankde)	Michael Gargiullo (mgargiullo)	Raphael Dunant (Racater)
Cesar Kohl (cesarkohl)	inaz0	Joerg Bruenner (JoergBruenner)	David Deatherage (securitydave)
John Carroll (yosignals)	Jim Fenton (jimfenton)	Matteo Pace (M4tteoP)	Sebastien gioria (SPoint42)
Steven van der Baan (vdbaan)	Jeremy Bonghwan Choi (jeremychoi)	craig-shony	Riccardo Sirigu (ricsirigu)
Tomasz Wrobel (tw2as)	Alena Dubeshko (belalena)	Rafael Green (RafaelGreen1)	mjang-cobalt
clallier94	Kevin W. Wall (kwwall)	Jordan Sherman (jsherm-fwdsec / deleterepo)	Ingo Rauner (ingo-rauner)

Dirk Wetter (drwetter)	Moshe Zioni (moshe-apiiro)	Patrick Dwyer (coderpatros)	David Clarke (davidclarke-au)
Takaharu Ogasa (takaharuogasa)	Arkadii Yakovets (arkid15r)	Motoyasu Saburi (motoyasu-saburi)	leirn
wet-certitude	timhemel	RL Thornton (thornshadow99)	Thomas Bandt (aspnetde)
Roel Storms (roelstorms)	Jeroen Willemsen (commjoen)	anonymous-31	Kamran Saifullah (deFr0ggy)
Steve Springett (stevespringett)	Spyros (northdpole)	Hans Herrera (hansphp)	Marx314
CarlosAllendes	Yonah Russ (yruss972)	Sander Majers (sanmai-NL)	Luboš Bretschneider (bretik)
Eva Sarafianou (esarafianou)	ataseren	Steve Thomas (Sc00bz)	Dominique RIGHETTO (righettod)
Steven van der Baan (svdb-ncc)	Michael Vacarella (Aif4thah)	Tonimir Kisasondi (tkisason)	Stefan Streichsbier (streichsbaer)
hi-unc1e	sb3k (starbuck3000)	mario-platt	Devdatta Akhawe (devd)
Michael Gissing (scolytus)	Jet Anderson (thatsjet)	Dave Wichers (davewichers)	Jonny Schnittger (JonnySchnittger)
Silvia Väli (silviavali)	jackgates73	1songb1rd	Timur - (timurozkul)
Gareth Heyes (hackvertor)	appills	suvikaartinen	chaals (chaals)
DanielPharos (AtlasHackert)	will Farrell (willfarrell)	Alina Vasiljeva (avasiljeva)	Paul McCann (ismisepaul)
Sage (SajjadPourali)	rbsec	Benedikt Bauer (mastacheata)	James Jardine (jamesjardine)
Mark Burnett (m8urnett)	dschwarz91	Cyber-AppSec (Cyber-AppSec)	Tib3rius
BitnessWise (bitnesswise)	damienbod (damienbod)	Jared Meit (jmeit-fwdsec)	Stefan Seelmann (sseelmann)
Brendan O'Connor (ussjoin)	Andrei Titov (andrettv)	Hans-Petter Fjeld (atluxity)	markehack
Neil Madden (NeilMadden)	Michael Geramb (mgeramb)	Osama Elnaggar (ossie-git)	mackowski